传统业务： RPC 、序列化、压缩等公共开销占 CPU 时间 22–27% ，客户感受到的是尾延迟； AI 化之后这些开销一项没少，又多了多级串行调用和向量检索两项

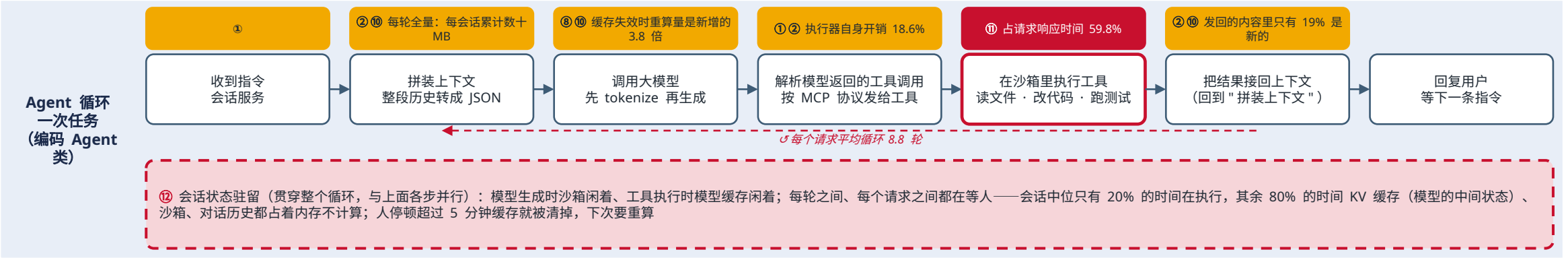


税（公共开销）	出现在	A： 占端到端时间的比例	B： 占 CPU 侧资源的比例	瓶颈在哪块硅	标注
① 网络收发与 RPC	传统 + AI 化编排	RPC 相关开销占请求延迟平均 2% ，最慢的请求可到 96% ； 每个服务旁挂代理（ sidecar ）的部署方式让 P99 延迟 +180% ； 大模型调用有 30–50% 延迟花在模型引擎之外的来回上	RPC 全路径占 7.1% CPU 时间（其中压缩 3.1 / TCP 1.7 / 序列化 1.2 / RPC 库 1.1 ）； 内核网络栈 18–20% ； 跑满 100 Gbps 需要 18 个核	CPU 核： 代码路径长，卡在取指令（前端）和单核速度	实测
② 序列化 / 反序列化	传统 + AI 化	逻辑简单的微服务里 RPC 层占 40–90% CPU 时间； AI 化后每次调用带的 prompt / 向量有 16–512 KB ，相当于传统 RPC 里最大的 1–10%	protobuf 3–4% （各类路径合计 9.6% ）； 实测成本的主体是拷贝，不是编码本身	CPU 核： 取指令 + 拷贝带宽	实测
③ 分配 · ④ 拷贝 · ⑤ 压缩 · ⑥ 哈希 / 加密	传统	（影响吞吐，对单次请求延迟影响小）	内存分配 6–7% （最大单项）、 memcpy 4–5% （网络栈里拷贝 >50% ，多数拷贝 <512 B ）、压缩 5–6% 、哈希 1–2% 、加密 <0.2%	CPU 核； 拷贝还吃内存带宽； 分配卡在 cache/TLB	实测
⑦ 向量检索 / embedding 表查找	AI 化新增（检索、推荐精排）	超大规模检索 + 小模型 + 短序列时占 86% ； 长上下文场景 <1% ； RAG 问答里非模型部分 >50% ； 推荐精排由 embedding 表的随机读主导	检索是 SIMD 扫描， 18 GB/s/ 核、内存带宽用到 80% ，已接近上限； embedding 表 TB 级、随机访问； 放不进内存后 SSD 上的索引要膨胀 5.8–7.7×	内存带宽 + 内存容量（不是核数）	实测

税的编号按流程中首次出现的顺序： ① 网络收发与 RPC ② 序列化 / 反序列化（数据格式转换） ③ 内存分配 ④ 内存拷贝与数据搬移 ⑤ 压缩 ⑥ 哈希与加密 ⑦ 向量检索与 embedding 表查找 ⑧ tokenize （把文本切成 token ） ⑨ 主机侧调度与任务下发 ⑩ 上下文全量重处理 ⑪ 沙箱与工具执行 ⑫ 会话状态驻留（占内存、不计算） ⑬ 训练数据加载与预处理。黄色标签 = 该环节上有税，标签内是税的编号和它占端到端时间的比例；红色标签 + 红框 = 该流程里占比最重的一到两个环节。

数据来源： Kanev 等 ISCA 2015 （ Google 全集群剖析）； Seemakhupt 等 SOSP 2023 （ Google RPC ）； Huye 等 ATC 2023 （ Meta 微服务）； Luo 等 SoCC 2021 （阿里）； mRPC NSDI 2023 ； Cai 等 ISCA 2023 ； RAGO ISCA 2025 ； Gupta 等 HPCA 2020 / RecNMP ISCA 2020 （ DLRM ）； Parrot OSDI 2024 ； HSTU ICML 2024 ； OneRec 2025 ； Cheng 等 2024 。

AI Agent： 一次任务里工具 / 沙箱执行占响应时间 59.8%，比模型生成（41%）还多；每一轮都把全部上下文重新处理一遍、会话 80% 的时间在等人，是另外两项大开销

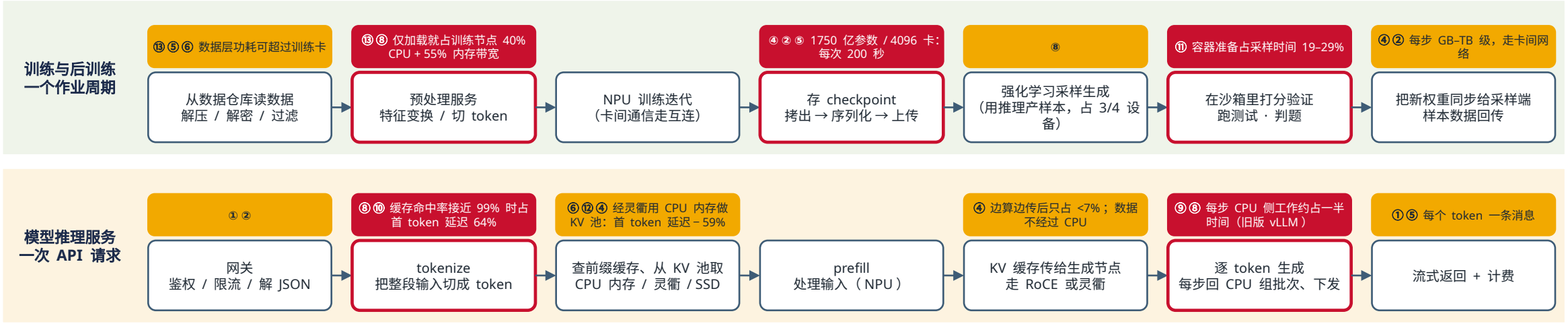


业务指标：每个任务 4.3 分钟（中位 38 秒）、8.8 次模型调用 + 10.8 次工具调用、$9.70（中位 $0.61）；每轮中位读入 11.9 万 token 的历史、只新增 875 个、输出 214 个；缓存命中 95.7%，但人的停顿让 59.4% 的会话至少被清一次缓存，占 31.5% 费用；费用的 59.5% 花在读缓存、只有 11.2% 在输出；最常用的三个工具（Bash / 读文件 / 改文件）占 >80% 调用，都是系统调用和文件 I/O 密集型；沙箱内存中位 0.8 GB、编译 / 跑测试时峰值 28 GB。

税（公共开销）	A：占端到端时间的比例	B：占 CPU 侧资源的比例	瓶颈在哪块硅	标注
⑪ 沙箱与工具执行	工具执行占请求响应时间 59.8%，模型生成 41.0%；超过 1 分钟的工具调用只占 4% 却占 85% 的工具时间；10 个 Agent 应用里 5 个的时延由非模型部分主导	沙箱内存中位 0.8 GB、峰值 28 GB；隔离层把系统调用放大：用户态内核（gVisor）下系统调用慢 ≥ 2.2 倍、打开文件慢 216 倍、读小文件慢 11 倍；从快照恢复的沙箱因逐页缺页执行慢 95%；本地 Agent 已改用进程级沙箱，云端仍是容器 / 微虚拟机	虚拟化路径（二级页表、页拷贝 / 清零、系统调用拦截）+ 内存容量 + 每台机器能装多少个沙箱	实测
⑩ 上下文全量重处理 + ⑧ tokenize	缓存命中 95.7% 但一旦失效，重算的 token 是真正新增的 3.8 倍；缓存命中率接近 99% 时，tokenize 占首 token 延迟 64%；CPU 核数配得不够时首 token 延迟高 1.47–7.11 倍	每轮把整段历史重新转 JSON、重新切 token（每核每秒只能切 2–4 MB 文本，且是串行算法）：每个请求约 4.2 MB 文本被重复处理（推算）；一次调用中位只新增 1.4K 字符，却把 10 万 token 的全文重切一遍	CPU 单核（分支和哈希密集，向量指令帮不上）+ 前端 CPU 核数	实测 + 推算
①② 工具调用协议与编排	Codex 执行器在模型和工具之外的自身开销 18.6%（平均 1.11 秒、最慢 1% 达 10 秒）；模型调用 30–50% 延迟在引擎之外	每会话累计数十 MB 序列化（一次请求要传 >20 MB 的 FP32 向量、每会话 >10 MB 文本来回）；MCP 协议 = JSON-RPC，每条消息一次 HTTP 请求，二进制内容用 base64 转文本	CPU 核：取指令 + JSON 解析 + 内核网络栈	实测
⑫ 会话状态驻留	（不占时延，占内存和费用）	会话中位只有 20% 的生命周期在执行；5 分钟缓存有效期导致 31.5% 的费用花在重算上；模型规划期间把闲着的沙箱搬出内存，平均省 4.6 倍（受控实验，不是等人时的空闲）；生产 Agent 的缓存复用率 >80%（人类聊天 54–62%）	内存容量 + 分层存储 + 恢复速度	实测

数据来源：TraceLab（华盛顿大学，2026，Claude Code + Codex 真实使用 4,300 个会话）；AgentSysBench（港科大，2026，含 35K 个生产会话）；SMetric（上交大 / 阿里百炼，2026）；TokTier（2026）；Chung 等 2026（CPU 核数不足的影响）；Firecracker NSDI 2020；gVisor HotCloud 2019；REAP ASPLOS 2021；Anthropic 沙箱工程博客 2025。

训练与推理：训练里让 GPU 等的是数据加载与 checkpoint，强化学习让沙箱创建最频繁；推理里留在 CPU 上的 tokenize 与每步调度占首 token 延迟最高可达 64%



税（公共开销）	A：对时延 / 训练进度的影响	B：占 CPU 侧资源的比例	瓶颈在哪块硅	标注
⑬ 训练数据加载与预处理（含 ⑥ 解密、② 反序列化）	数据供不上会直接让 GPU 停下；评估作业 29.5% 的时间花在加载模型和切 token；预处理需求两年增长 3.5 倍	训练节点仅 " 加载已预处理好的数据 " 就占 40% CPU + 55% 内存带宽，成分是 TLS 解密、Thrift 反序列化、网络栈；TLS 让内存带宽消耗放大 3 倍；同时训练节点自身的 CPU 是闲的（16 核 /GPU、内存用了不到一半）	内存带宽 + CPU 核（每一步都要把数据过一遍内存）	实测
④ ② ⑤ checkpoint	1750 亿参数模型、4096 卡，存一次 checkpoint 到 HDFS 端到端 200 秒，远超一次迭代；同步等待会拖慢训练最多 43%	从卡拷到内存 → 序列化 → 写共享内存 → 上传，全在主机 CPU 上；单文件 2–3 GB/s 的上限来自 CPU 序列化而不是网卡；异步化要靠 CPU 内存放得下好几份 checkpoint	拷贝 + 序列化 + 内存容量	实测
⑪ 强化学习的沙箱	容器化环境准备约 89 秒，64–256 个并行时占采样时间 19–29%；改进程级隔离后约 24 秒、占 8–9%，每条采样时间缩短 15–29%	每个任务的镜像 1.2–2.5 GB → 进程级存储约为原来的 5%；生产沙箱引擎 100 ms 启动、每秒 5,000 次；同步采样要等最慢的样本，改异步后快 2.77 倍	虚拟化路径 + 内存 + 沙箱密度（同 Agent，但无停顿、用完即弃、并发上万）	实测
⑧ ⑩ 推理前端 tokenize	缓存命中率接近 99% 时 tokenize 占首 token 延迟 64%；16 核前端每秒 40 个请求就饱和；CPU 核数不够时首 token 延迟高 1.5–7 倍	切 token 每核每秒 2–4 MB、串行算法；前端把整段输入重切一遍，而 NPU 只算新增部分	CPU 单核速度 + 每张 NPU 配的前端核数	实测
⑨ 主机侧调度与任务下发	旧版 vLLM 每生成一个 token，CPU 侧工作约占一半时间；昇腾上整图下发一次 0.6–0.8 ms；与其它程序同机运行时首 token 延迟恶化 139 倍	CPU 做的事：整理批次元数据、准备张量、把 token 转回文字；页表查找占末级缓存等待的 85%；Arm 核下发任务比 x86 慢 17–22%；GPU 越快，CPU 拖后腿的区间越大	地址翻译（TLB）/ 末级缓存 / 单核速度 + CPU 到 NPU 的链路；对同机干扰极敏感	实测
④ ⑫ KV 缓存传输与 KV 池	KV 缓存边算边传后只占输入处理时间 <7%；把 CPU 内存经灵衢池化作 KV 池后，输入处理吞吐 +52%、首 token 延迟 – 34% ~– 59%	每个 token 的 KV 缓存 69–320 KB（DeepSeek-V3 / Llama-70B）；2 TB 内存 ≈ 310 个 10 万 token 的会话（推算）；生产系统一律让数据不经过 CPU 核（GPUDirect / RoCE / 灵衢内存语义）	互连带宽与访问方式 + 内存容量（不是核）	实测 + 推算

数据来源：Zhao 等 ISCA 2022（Meta 训练数据摄取）；Acme NSDI 2024；ByteCheckpoint NSDI 2025；MegaScale NSDI 2024；AReaL 2025；SWE-MiniSandbox 2026；Ring-1T 2025；TokTier 2026；Chung 等 2026；WukLab 2024 / NanoFlow OSDI 2025；Blink 2026；GH200 刻画 2025；CloudMatrix384 2025；xDeepServe 2025；Splitwise ISCA 2024；DistServe OSDI 2024；Mooncake FAST 2025。

规划：软件先做沙箱、 tokenize 、 RPC/ 序列化三项并测出鲲鹏自己的开销分解；硬件投取指令与地址翻译、内存系统、灵衢数据路径和沙箱辅助， KAE 直通式接数据层

#	税（占比）	瓶颈在哪块硅	软件动作（现有硬件上先做）	硬件动作（下一代架构 / 产品配置）	验收指标
1	⑪ 沙箱与工具执行（ Agent 59.8% ； 强化学习采样 19–29% ）	虚拟化路径 + 内存容量 + 沙箱密度	进程级沙箱替代容器；快照恢复时预取工作集；预热池；分层文件系统；用 virtCCA 做机密虚拟机；推理主机和沙箱不混部	CPU ： 二级页表复制、内存清零卸载、快照恢复走 DMA 、低开销机密计算、超线程隔离；产品：高核数款 + 大内存	每核能装的沙箱数；创建 / 恢复延迟（对标 100 ms / 每秒 5,000 次）；机密虚拟机 I/O 开销（对标 <29.7% ）
2	⑧ tokenize + ⑩ 上下文全量重处理（占首 token 延迟 64% ； 重算量 3.8 倍）	CPU 单核速度 + 前端核数	只切新增部分的增量 tokenize + 缓存切分结果 + 增量序列化；按每张 NPU 配前端核数；用独立 CPU 池承接前端	CPU ： 单核性能；产品：每张 NPU 配的前端核数（加核是最便宜的缓解）	首 token 延迟里 tokenize 占比随缓存命中率的曲线；前端每秒请求数的饱和点
3	① RPC / ② 序列化（模型调用 30–50% 在引擎外；传统 7.1% CPU 时间、尾延迟 96% ）	取指令（一级指令缓存 / 分支预测 / 取指带宽） + 拷贝	按热点重排代码布局（ PGO/BOLT 、指令大页）；零拷贝序列化（ protobuf 字段直接作为灵衢发送单元）； bRPC- 灵衢 默认改无连接模式 + 小消息直写 + 免 TCP 握手；服务端编排减少来回	CPU ： 一级指令缓存、取指带宽、分支预测（指令缓存减到 1/4 的 SKU 能效低 57% ）； KAE ： 结构化数据转换引擎（ protobuf + JSON ）； 灵衢：按消息标优先级、完成事件带时间戳、可编程拥塞控制、网卡侧按核负载分发	取指令等待比例、指令缓存缺失率；每核每秒请求数、往返延迟、连接数与延迟的关系（对标 eRPC 每核 1000 万次 / 秒、 Falcon 10 万连接不掉速）
4	⑦ 向量检索 / embedding 表（ <1%–86% ；推荐卡内存）	内存带宽 + 内存容量	faiss / SVS / ScaNN 移植到 SVE 向量指令并测带宽利用率；向量压缩（ LVQ/PQ ）；推荐 embedding 表读取的访存优化	CPU ： 内存通道数 / 带宽 / 容量；矩阵指令扩展（ SME2 ）让 embedding 、重排模型跑在 CPU 上；灵衢：内存池作为第二层内存（小粒度随机读比 SSD 合适）；不做专用距离计算引擎	带宽利用率（对标每核 18 GB/s 、 80% ）；放不进本机内存后经灵衢远端读的延迟
5	⑬ 训练数据层 / ④②⑤ checkpoint （ 40% CPU + 55% 带宽；每次 200 秒）	内存带宽 + 拷贝 + 内存容量	KAE 的压缩 / 加密以 " 数据流经引擎、不额外落一次内存 " 的方式接入预处理、加载和 checkpoint 流水；给 UADK 做 Python/PyTorch 零拷贝接口；预处理和训练节点分开部署	KAE ： 小数据块的固定开销、直通式 DMA ； 片上： DPA 做数据搬移（类 DSA ）；产品： CPU 密集型节点（按数据层 / 训练节点 / 沙箱池分开配比）	不多过一次内存的约束下的实际收益；checkpoint 停顿时间；数据层每张卡需要的 CPU 核数
6	⑨ 主机侧调度与任务下发（每步 CPU 约占一半；整图下发 0.6–0.8 ms ； 同机干扰 139 倍）	地址翻译 / 末级缓存 / 单核 + CPU 到 NPU 链路	整图下发、异步调度、采样和元数据搬到 NPU 、大页、推理主机与沙箱不混部	CPU ： 更大更快的二级 TLB 、页表缓存、单核性能；灵衢：用内存语义队列下发任务、更低的下发延迟（对标 x86 2.3 μs ）	每步 CPU 时间分解；鲲鹏到昇腾的整图下发延迟；同机干扰下首 token 延迟恶化倍数
7	④ KV 缓存传输 / ⑫ 会话状态驻留（传输 <7% ； 首 token 延迟 – 59% ； 会话 80% 空闲）	互连访问方式 + 内存容量	实测已在 Mooncake 主线里的灵衢后端（鲲鹏侧 kunpeng_transport / 昇腾侧 ubshmem ）；参考 EMS 做 KV 池；空闲会话搬出高速内存、延长缓存有效期、内存压缩（ zswap ）	灵衢： KV 直传零拷贝、 CPU 侧 32 lane 出口带宽、按缓存行粒度按需取； CPU ： 大内存； KAE ： 压缩空闲状态、内存侧数据变换 + 压缩（ KV 无损压 46.9% ）	每核每秒请求数 / 带宽 / 远端读延迟；每台机器能托管的会话数（ 2 TB ≈ 310 个 10 万 token 会话）
8	③⑤⑥ 传统公共开销延续（ 22–27% CPU 时间，十年稳定）	CPU 核（取指令、 cache/TLB ）	换更快的内存分配器（ mimalloc / TCMalloc ） + 大页； KAE 压缩接入 RPC 库和文件格式；建立类似 DCPerf 的基线测试	CPU ： 取指令 + 地址翻译（同第 3 行）； KAE ： 把小数据块压缩的固定开销压到贴核方案的 2 倍以内	类似 DCPerf 的开销分解；压缩在 1–64 KB 数据块上的盈亏点

三条原则：① 开销卡在哪块硅，就在那块硅上解决——取指令 / 地址翻译 / 单核对应老开销和主机侧调度，内存系统与灵衢对应检索和会话状态，搬移 / 转换 / 压缩引擎对应拷贝 / 序列化 / 压缩，虚拟化路径对应沙箱；② 软件先把开销测出来、把能用软件消掉的消掉（前三行的软件项都能在现有硬件上落地），剩下的才是硬件目标；③ 灵衢的三个卖点（无连接、小消息直写、免握手建连）在现在的默认配置里是关着的，先打开再测——灵衢目前公开的性能数字只有超节点级 2.1 μs 一个，而 AI 数据路径的 EMS / XCCL / Mooncake 三条都已在生产或主线代码里。

数据来源同前三页；灵衢机制依据灵衢协议文档、 URMA/UMQ 分析与 brpc_ub 代码；鲲鹏 920 微架构依据 Chips and Cheese 公开测试；鲲鹏 950/960 、 TaiShan 950 SuperPoD 、灵衢 2.0 依据 HC2025 发布。行序 = 优先级（按占端到端时间的比例）。